Ders 8 Genişletilmiş Notu: Uygulama: Poliklinik Sistemi İçin Kullanım Şeması, Kullanım Senaryoları ve Sınıf Şeması

Ders 8 Genişletilmiş Notu: Uygulama: Poliklinik Sistemi İçin Kullanım Şeması, Kullanım Senaryoları ve Sınıf Şeması

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Uygulamanın Amacı ve Vaka Metni
- 2. Kullanım Şeması (Use Case Diagram)
- 3. Kullanım Senaryosu: Tedavi Bilgisi Girişi
- 4. Kullanım Senaryosu: Ücret Ödeme
- 5. Sınıf Şeması (Class Diagram)
- 6. Ders Sonrası Notlar
- 7. Örnek Sorular

1. Uygulamanın Amacı ve Vaka Metni

Bu ders bir uygulama dersidir. Prof. Dr. Oya Kalıpsız dersi açtı ve vizeden önceki hafta uygulama yapılacağını, dönem boyunca görülen nesneye yönelik analiz ve tasarım şemalarının bir örnek üzerinde çizileceğini söyledi. Uygulamayı dersin asistanı anlattı. Öğrencilerin şemalarla ilgili aklına takılanları sınava hazırlık olarak bu sırada sorabileceği belirtildi.

Asistan, bir proje metni (case) üzerinden üç analiz çıktısı hazırlanacağını söyledi:

- Olası kullanıcı öykülerinden yola çıkarak bir kullanım şeması (use case diagram) çizmek.
- Bu şemadaki bir iki iş için kullanım senaryosu (use case scenario) yazmak.
- Sistemin sınıf şemasını (class diagram) çizmek.

Önce vaka metni birlikte okundu. Asistana göre bu, günlük hayattan iyi bildiğimiz bir sistemin "kabataslak" planıdır.

1.1 Çalışan yönetimi

Metin şöyle başlıyor: Yeni açılan bir poliklinik için bir hasta ve çalışan yönetimi yazılımına ihtiyaç vardır. Programın çalışan yönetimi bölümünde her çalışan için şu bilgiler saklanmalıdır:

- Ad, soyad, kimlik numarası.
- İşe başlama tarihi ve çalıştığı birim.
- Kişi işten ayrıldıysa ayrılma tarihi.
- Erişim (iletişim) bilgileri: ikamet adresi, iş telefonu ve cep telefonu.

Dört tür çalışan vardır: doktorlar, hemşireler, hizmetliler ve güvenlik görevlileri. Hepsinin ortak (sabit) bilgileri yukarıdakilerdir. Doktorlar için bunlara ek olarak diploma numarası da tutulur.

Doktorlar sistemde uygun günlerini, randevularını ve hastalarının tedavi işlemlerini yönetebilmelidir. Bir hasta için reçete oluşturmak ve tedavi bilgisi girmek gibi işleri sistemden yapabilmelidir. Bir doktor hastanede başhekim olarak da görev yapabilir. Başhekim hastalarla ilgilenmenin yanında hastanenin yönetiminden ve idari işlerden de sorumludur.

1.2 Hasta izleme

Hastalar için de ad, soyad, kimlik numarası ve erişim bilgileri (adres, telefonlar) tutulur. Sistemde yeni hasta kaydı oluşturma ve randevu işlemleri yapılabilmeli, bu kayıtlar görüntülenip güncellenebilmelidir. Hastalar da sisteme girip randevu alabilir; yani hasta da sistemin bir kullanıcısıdır.

Randevu günü gelince hasta polikliniğe gider ve bankoda ücretini öder. Burada bir ayrıntı vardır: Herkes için sabit bir ücret vardır, ama hastanın çalıştığı kurumun poliklinikle bir anlaşması varsa ödemede belli bir indirim yapılır. Hasta ödemeyi yaptıktan sonra kayıt görevlisi hastanın sistemdeki kaydını açar. Hasta bundan sonra muayene sırasını beklemeye başlar.

Hasta kaydında şu bilgiler tutulur: hastanın başvurduğu birim, tedavinin başladığı zaman, doktor, tanı ve ilaçlar.

2. Kullanım Şeması (Use Case Diagram)

Asistan çizimleri Violet UML Editor ile yaptı ve önce kullanım şemasını seçti. Kullanım şemasına öğrenciler arasında “çöp adamlı UML” dendiğini söyledi.

2.1 Kullanım şemasının öğeleri

Asistan önce şemanın yapı taşlarını hatırlattı:

- **Aktör (actor):** Sistemi kullanacak kişiler. Çöp adamla çizilir.
- **Kullanım senaryosu (use case):** Aktörün sistem üzerinde yaptığı bir iş, bir aktivite.
- **Etkileşim (interaction):** Bir aktör ile bir use case arasındaki bağlantı.
- **extend:** Bir use case başka bir use case ile genişletilebilir; işe ek bir özellik eklenir ya da iş bazı durumlarda farklı bir akışa girer.
- **include:** Asistanın tanımıyla, bir use case’in gerçekleşmesi için başka bir use case’in de olması gerekir. “Bu iş bu işi include eder” denir. Asistan bunu “bu, bundan önce olmuş olmalıdır” mantığıyla açıkladı. (Bu tanım için bkz. Bölüm 6.)
- **Genelleştirme (generalization):** Bir aktörün daha özel bir alt aktörü tanımlandığında, üstteki aktör alttakinin genelleştirmesidir.

2.2 Hasta ve kayıt görevlisi

Çizime hasta aktörüyle başlandı. Asistan, hastanın sistem üzerinden yapabildiği işleri tek tek use case olarak ekledi ve her birine hastadan bir etkileşim çizgisi çekti:

- **Randevu Alma:** Hasta randevusunu alır.
- **Tedavi Bilgileri Görüntüleme:** Hasta, kendisi için sisteme girilmiş tedavi bilgilerini görüntüler.
- **Ücret Ödeme:** Randevu günü polikliniğe gelen hasta ücretini öder.

Ardından metindeki iki işin sırasına dikkat çekildi: Önce hasta ücreti ödüyor, sonra kayıt görevlisi kaydı açıyor. Bunun için ikinci bir aktör olarak **Kayıt Görevlisi** eklendi. Kayıt görevlisi **Hasta Kaydı** use case’i ile etkileşim içindedir; hastanın girişini yapar.

Asistan bu iki iş arasında bir `include` ilişkisi kurdu: Hasta kaydının açılabilmesi için ücret ödemesinin önceden alınmış olması gerektiği için “Hasta Kaydı, Ücret Ödeme’yi include eder” dedi. Ok, Hasta Kaydı’ndan Ücret Ödeme’ye çizildi.

Sonra anlaşmalı kurum indirimi ele alındı. Ücret ödemede herkese sabit bir ücret çıkarılır; ama hastanın çalıştığı kurumla polikliniğin anlaşması varsa ödeme belli bir indirim oranıyla alınır. Yani ücret ödemenin alternatif bir durumu vardır. Asistan, “bu use case’in bir tık farklı bir durumu olabilir” denildiğinde `extend` kullanıldığını söyledi. **Anlaşmalı Kurum İndirimi**

adında yeni bir use case eklendi ve bunun Ücret Ödeme'nin genişletilmiş (extend edilmiş) hâli olduğu gösterildi.

> **Hocanın vurgusu:** Buradaki `extend`, nesneye yönelik programlamadaki kalıtım (birbirinden kalıtımla türetme) değildir. Yalnız bu işin bir tık genişlediğini, bir durum oluşursa alternatif bir akışa girdiğini gösterir.

2.3 Hekim

Asistan "bundan sonraki en önemli aktör hangisi?" diye sordu ve hekimle devam edildi. Hekimin işleri şöyle kategorilere ayrıldı:

- **Hasta Bilgisi Görüntüleme**
- **Tedavi Bilgisi Girişi:** Hekim hastanın tedavisiyle ilgili bilgileri girer.
- **Randevu Görüntüleme:** Hekim kendi günlük randevularını, programını görüntüler.

Sonra asistan, hastanın Tedavi Bilgileri Görüntüleme işi ile hekimin Tedavi Bilgisi Girişi işi arasında bir bağlantı kurulabileceğini söyledi: Hastanın tedavi bilgisini görüntüleyebilmesi için hekimin önce tedavi bilgisi girişini yapmış olması lazım. Öğrencilere bunun `extend` mi `include` mu olduğu soruldu ve cevap `include` olarak verildi: "Tedavi bilgisi görüntüleme, tedavi bilgisi girişini include eder." (Bu kararın UML açısından değerlendirmesi Bölüm 6'dadır.)

Daha sonra, tedavi bilgisi girişine bir ekleme yapıldı. Hasta çoğu zaman ayaktan tedavi alıp gider; ama doktor bazen hastanın durumunu ağır görür ve yatarak tedavi görmesine karar verir. Bu, tedavi bilgisi girişi sırasında verilebilecek ek bir karardır. Bu yüzden **Yatarak Tedavi** use case'i eklendi ve Tedavi Bilgisi Girişi'nin `extend`'i olarak gösterildi. Asistanın genel kuralı: "Bu iş yapılırken böyle bir karar verilebilir, bu iş de yapılabilir" dediğimiz alternatif akış durumlarında `extend` kullanılır.

2.4 Başhekim ve aktör genelleştirmesi

Metne göre hekimin daha özel, kendine ait görevleri olan bir hâli vardır: başhekim. Bu yüzden **Başhekim** ayrı bir aktör olarak eklendi ve Hekim'e genelleştirme ilişkisiyle bağlandı. Hekim daha genel hâl, Başhekim daha özel hâldir. Başhekim, hekimin yaptığı her şeyi yapar; ek olarak hastane yönetiminden sorumludur. Başhekime özgü use case'ler:

- **Planlama Yapma:** Çalışanlarla ilgili planlamalar.
- **Çalışan Bilgisi Görüntüleme**

2.5 Güvenlik görevlisi ve beyaz kod

Asistan, ana metinde görevini net yazmadığı bir aktörü de eklemek istediğini söyledi: **Güvenlik Görevlisi**. Hastanelerde doktorların beyaz kod verme durumu vardır. Bir hasta ya da hasta yakını güvenliği tehdit eden bir durum oluşturursa hekim kod verir; güvenlik görevlisi bu uyarı koduyla odaya gelir ve müdahale eder. Şemada hekim ile güvenlik görevlisi arasında bu etkileşim gösterildi: Hekim beyaz kodu verir, güvenlik görevlisi beyaz kodu alır ve gereken işi yapar.

2.6 Şemanın metin hâli

Derste çizilen kullanım şeması, altyazıdan okunabildiği kadarıyla şöyledir:

```
Aktörler: Hasta, Kayıt Görevlisi, Hekim, Başhekim, Güvenlik Görevlisi

Hasta ------------ (Randevu Alma)
Hasta ------------ (Tedavi Bilgileri Görüntüleme)
Hasta ------------ (Ücret Ödeme)
Kayıt Görevlisi -- (Hasta Kaydı)

(Hasta Kaydı) ---------------- <<include>> ---> (Ücret Ödeme)
(Anlaşmalı Kurum İndirimi) --- <<extend>>  ---> (Ücret Ödeme)

Hekim ------------ (Hasta Bilgisi Görüntüleme)
```

```
Hekim ------------ (Tedavi Bilgisi Girişi)
Hekim ------------ (Randevu Görüntüleme)

(Tedavi Bilgileri Görüntüleme) - <<include>> ---> (Tedavi Bilgisi Girişi)
(Yatarak Tedavi) ------------- <<extend>>  ---> (Tedavi Bilgisi Girişi)

Başhekim ----|> Hekim                (aktör genelleştirmesi)
Başhekim --------- (Planlama Yapma)
Başhekim --------- (Çalışan Bilgisi Görüntüleme)

Hekim ----- (beyaz kod) ----- Güvenlik Görevlisi
```

Asistan özetle şunu söyledi: Kullanım şemasında aktörler, use case'ler ve etkileşimler vardır; use case'ler arasında da `extend` ve `include` ilişkileri bulunabilir.

> **Ek bilgi (derste anlatılmadı):** UML'de ok yönleri sabittir. `include` oku temel use case'den içerilen use case'e, `extend` oku ise genişleten use case'den temel use case'e doğru çizilir. Aktör genelleştirmesinde içi boş üçgen uçlu ok, özel aktörden genel aktöre gider. Derste çizilen oklar bu yönlerle uyumludur.

3. Kullanım Senaryosu: Tedavi Bilgisi Girişi

Sorunun ikinci kısmı, doktorun bir hastaya ait tedavi bilgisi girişi için kullanım senaryosu yazılmasını istiyordu. Asistan, şemadaki her use case için böyle bir senaryo yazılabileceğini söyledi: Senaryoda işin aktörü kim, bu işle kimler ilgileniyor, akış nasıl işliyor, madde madde yazılır. Derste iki senaryo yazıldı: Tedavi Bilgisi Girişi ve Ücret Ödeme. Senaryolar hazır bir tablo şablonu doldurularak yazıldı.

3.1 Başlık alanları

- **Senaryo adı:** Use case'in adı doğrudan yazılır: Tedavi Bilgisi Girişi.
- **Birincil aktör:** İşi doğrudan yapan, işle doğrudan ilgili olan kişi: Hekim.
- **İlgililer ve beklentileri:** Bu işten doğrudan ya da dolaylı etkilenen kişiler ve bu işten ne istedikleri. Birincil aktör her zaman ilk ilgilidir.
  - Hekim: Hasta bilgilerine ulaşmak ve kararlaştırdığı tedaviye dair bilgileri sisteme girmek ister.
  - Hasta: Sistemden bilgilerini görebilmek ve reçetesine ulaşabilmek ister.

3.2 Ön koşullar ve son koşullar

Ön koşul, bu iş başlamadan önce sağlanmış olması gerekenlerdir. Tedavi bilgisi girebilmek için hekimin önce sisteme girmesi gerekir:

- Hekim, kendine ait kullanıcı adı ve şifresiyle sisteme giriş yapmıştır.
- Hekim, ilgili hastanın kimlik bilgisini (örneğin kimlik numarası ya da ad soyad) girerek hastanın ekranına ulaşmıştır.

Bu koşullar sağlanınca hekim artık hastanın ekranındadır ve tedavi bilgisi girebilir.

Son koşul, "bu da yapılmışsa iş bitmiştir" dediğimiz durumdur:

- Reçete oluşturulmuştur.
- Planlanan tedaviye dair girilen bilgiler kaydedilmiştir.

3.3 Ana akış

Asistan ana senaryoyu adım adım yazdı. Hekim sisteme girmiş, hastanın ekranına ulaşmıştır. Ekranda "tedavi bilgisi girişi yap" gibi bir modül ya da buton vardır.

- 1. Hekim, hasta için tedavi bilgisi girişi yapmayı seçer.
- 2. Hekim, açılan ekranda teşhis ettiği hastalığı belirtir.
- 3. Hekim reçeteyi hazırlar: reçete için ilaçları ve dozlarını girer.
- 4. Sistem reçeteyi onaylar ve ilgili reçete hastaya verilir.

Onay adımının amacı şudur: Bazı ilaçları alabilmek için ayrıca rapor gerekir; kişinin örneğin eczaneye bir rapor götürmesi gerekebilir. Hekim son aşamada onayla dediğinde sistem, bu reçete için rapor gerekip gerekmediğini bildirir.

3.4 Alternatif akışlar

Alternatif akış, ana akıştaki bir adımla ilgili yaşanabilecek bir sorun ya da yapılabilecek ek bir iştir. Asistan bunu, ana senaryodaki bir adımın altına bir alt başlık açmak gibi düşünmeyi önerdi: "Bu da olursa şu yapılsın."

İlk alternatif, 2. adımla ilgilidir. Teşhis edilen hastalık kaydedilirken doktor, kullanım şemasında gösterildiği gibi yatarak tedaviye karar verebilir. Normal durum (varsayılan) ayaktan tedavidir.

- **2a.** Hastanın durumu ağırdır.
  - 1. Hastanın yatarak tedavi alması kararlaştırılır.

Asistan, bu akış daha ayrıntılı olsaydı alt maddelerin 2, 3 diye devam edeceğini, ama iş tek maddede bitiyorsa başka madde yazmaya gerek olmadığını söyledi. Aynı adım için başka bir durum da olsaydı (asistanın örneği: hastanın refakatçisi yoktur) bu **2b** olarak yazılır ve altına yapılacak işler sırayla eklenir. Yani bir adım için A durumu, B durumu, C durumu ayrı ayrı yazılır.

İkinci alternatif, 4. adımla ilgilidir:

- **4a.** Reçetedeki ilaçlardan biri ya da birkaçı için özel rapor gerekiyordur.
  - 1. Hekim raporu hazırlar.
  - 2. Hekim raporu sisteme kaydeder.

  Hocanın vurgusu: Alternatif akışta yazılan durumların kullanım şemasında da `extend` olarak gösterilmesi en doğrusudur. Yatarak tedavi şemada gösterilmişti. Asistana göre özel rapor hazırlanması da Tedavi Bilgisi Girişi'ne bağlı bir `extend` use case'i olarak gösterilmeliydi. Çünkü hem use case'den hem senaryodan çıkarım yapılır: "Bu iş böyle genişleyebiliyor, başka bir akışa girebiliyor."

4. Kullanım Senaryosu: Ücret Ödeme

Senaryo yazmayı pekiştirmek için ikinci bir senaryo, Ücret Ödeme için yazıldı.

4.1 Başlık alanları

- **Senaryo adı:** Ücret Ödeme.
- **Birincil aktör:** Hasta (öğrencilere soruldu ve hasta cevabı alındı).
- **İlgililer ve beklentileri:** Hasta zaten sabit bir ilgilidir. Öğrencilere başka hangi aktörün ilgili olduğu soruldu; cevap kayıt görevlisiydi.
  - Hasta: Ödemesini yapıp muayene olmak ister.
  - Kayıt görevlisi: Hastanın ödemesini alıp sisteme işlemek ister.

4.2 Ön koşullar ve son koşullar

Ön koşullar:

- Hasta randevu almıştır.
- Hasta randevu günü polikliniğe (fiziksel olarak) gelmiştir.

Son koşullar, işin doğru şekilde tamamlandığını gösteren durumlardır:

- Ödeme başarıyla alınmış ve fatura hazırlanmıştır.
- Hasta muayene sırasına alınmıştır.

4.3 Ana akış

- 1. Sistem, hastanın aldığı randevuya göre ödenecek tutarı gösterir.
- 2. Hasta ödemeyi yapar.

- 3. Sistem, ödemenin yapıldığına dair faturayı oluşturur.

Asistan, tutarın randevuya göre değişebileceğini örnekledi: Ücret, doktorun unvanına göre farklı olabilir; profesör ayrı, doçent ayrı.

Özetle ana akış: Ücretimi gördüm, ödememi yaptım, faturamı aldım.

4.4 Alternatif akışlar

İlk alternatif, kullanım şemasında `extend` olarak gösterilen anlaşmalı kurum durumudur:

- **2a.** Hasta, poliklinikle anlaşmalı bir kurumda çalışıyordur.
  - 1. Belirlenen indirim oranı uygulanarak ücret alınır.

Asistan, aynı adım için başka bir durum oluşursa nasıl gösterildiğini anlatmak amacıyla ikinci bir alternatif ekledi:

- **2b.** Kartla ödeme alınırken sorun oluşur.
  - 1. Ödemenin nakit yapılması istenir.

Kural yine aynıdır: Bir adım için bir durum A ile, aynı adım için bambaşka bir durum B ile yazılır; her birinin altına akışı 1, 2, 3 diye verilir. Burada her iş tek maddede yazılabildiği için 2. ve 3. maddelere gerek kalmadı.

> **Hocanın vurgusu:** Bir kullanım senaryosunda mutlaka şunlar yazılmalıdır: birincil aktör, ilgililer ve beklentileri, ön koşullar, son koşullar, ana akış ve alternatif akışlar.
>
> **Ek bilgi (derste anlatılmadı):** Derste kullanılan şablon, büyük olasılıkla Craig Larman'ın "Applying UML and Patterns" kitabındaki ayrıntılı (fully dressed) kullanım senaryosu biçimine dayanır. Bu biçimde ana akışa "ana başarı senaryosu" (main success scenario), alternatif akışlara "uzantılar" (extensions) denir. Uzantılar, ilgili ana adımın numarası ve bir harfle (2a, 2b, 4a) numaralanır.

5. Sınıf Şeması (Class Diagram)

Sorunun üçüncü kısmı sınıf şemasıydı. Asistan, sınıf şemalarının nesneye yönelik programlamadan bilindiğini, bir sistem gerçeklenirken sürekli sınıf şeması üzerinden gidildiğini, çünkü sınıfla ilgili pek çok bilginin bu şemada görülebildiğini hatırlattı.

5.1 Sınıf kutusu ve analist bakışı

Sınıf kutusu üç bölüme ayrılır:

- **Üst bölüm:** Sınıfın adı.
- **Orta bölüm:** Sınıf değişkenleri, yani nitelikler (attributes). Bu sınıftan üretilen bir nesnenin sahip olduğu özelliklerdir.
- **Alt bölüm:** Sınıfın metotları.

> **Hocanın vurgusu:** Sistemi şemaya dökerken analist tarafında olduğumuz için metot ayrıntısına girilmedi. Sınıf şemasında yalnız sınıf adları ve nitelikler gösterildi.

Niteliklerin önüne konan - işareti, niteliğin private olduğunu, yani yalnız sınıf içinden erişilebildiğini gösterir. + public demektir. Asistan, metotların genelde + ile public yazıldığını söyledi.

5.2 Çalışan ve Erişim sınıfları

İlk sınıf **Çalışan** oldu. Metne göre çalışanlar için göreve başlama tarihi ve görevden ayrılma tarihi tutulur. Asistan, kişi ayrılmamışsa ayrılma tarihinin zaten boş (null) bir değer olacağını belirtti.

Çalışanların ad, soyad, kimlik numarası, adres ve telefon bilgileri de tutulmalıdır. Ama aynı bilgiler hastalar için de tutulacaktır. Asistan bunları iki sınıfa ayrı ayrı yazmak yerine şu tasarımı yaptı: Bu bilgileri tutan ayrı bir **Erişim** (erişim/iletişim bilgileri) sınıfı tanımlanır. Hem Çalışan hem Hasta bu sınıfla bir ilişki (association) kurar ve onun bir nesnesini nitelik olarak kullanır. Böylece beş altı veri tek bir yerde toplanır.

Erişim sınıfının nitelikleri:

- Ad
- Soyad
- Kimlik numarası
- Telefonlar: Bir kişinin iş, cep, ev telefonu gibi birden çok telefonu olabileceği için bunlar bir dizide tutulur.
- Adres (ev adresi): String tipinde.

Çalışan sınıfına, Erişim tipinde "erişim" adlı bir nitelik yazıldı ve Çalışan ile Erişim arasına bir ilişki çizildi. Bir çalışanın bir erişim bilgisi olacağı için ilişki birebir kuruldu. Violet'ta ilişkinin üzerine çift tıklanarak başlangıç ucu (start label) ve bitiş ucu (end label) için çokluk girilebilir. Ortaya bir açıklama ("bundan nesne alır" gibi) yazılabilir, ama asistan yazmadı; yalnız birebir ilişkiyi gösterdi.

5.3 Kalıtım: çalışan türleri ve başhekim

Dört çalışan türü, **Hekim**, **Hemşire**, **Güvenlik Görevlisi** ve **Hizmetli**, ayrı sınıflar olarak eklendi. Asistan kalıtımı hatırlattı: Birden çok sınıf bir üst sınıfın aynı özelliklerini kullanacaksa, bu nitelikler her sınıfa tek tek yazılmaz. Bir üst sınıf yapılır ve alt sınıflar kalıtımla (inherits) onun bütün özelliklerini kullanır. Dört sınıfın hepsi Çalışan'dan kalıtımla türetildi.

Metne göre doktorların ayrıca diploma numarası vardır. Bu yüzden yalnız Hekim sınıfına ek bir nitelik yazıldı: diploma numarası, integer tipinde. Nitelikler ad ve tip ile verilir.

Sonra **Başhekim** sınıfı eklendi ve Hekim'den kalıtımla türetildi. Asistanın gerekçesi: Şemada metotlar gösterilmese de sınıfların yaptığı işler metotlarında bulunur. Başhekimin hekimden ayrı işleri olduğu için ayrı bir sınıf olarak gösterilmelidir.

5.4 Hasta, Randevu ve "gizli bilgi"

Hasta sınıfı eklendi ve Erişim ile birebir ilişki kuruldu: Bir hastanın bir erişim bilgisi olur. Bu kez asistan Hasta sınıfına "erişim" niteliğini yazmadı.

> **Hocanın vurgusu:** İki sınıf arasındaki ilişki şemada görünüyorsa, Hasta'nın bir Erişim nesnesi kullandığı zaten şemadan çıkarılabilir. Asistan buna "gizli bilgi" (hidden information) dedi. Bu niteliği ayrıca yazmak ya da yazmamak isteğe bağlıdır.

Hasta randevu alır. Bu yüzden randevularla ilgili bilgilerin tutulduğu bir **Randevu** sınıfı eklendi. Bir hastanın birden çok randevusu olabileceği için ilişki bire çok kuruldu: Başlangıç ucu 1, bitiş ucu *. Asistan * için "bir veya birden çok" dedi (bkz. Bölüm 6).

5.5 Reçete ve İlaç: aggregation

Hasta ile hekim arasında, tedaviyle ilgili bilgilerin (ilaçlar, reçeteyi kimin, nasıl oluşturduğu) görüntülenebildiği bir **Reçete** sınıfı eklendi. Nitelikleri:

- Reçete numarası
- Düzenleyen: Hekim tipinde
- Düzenlenme tarihi
- Hasta: Reçetenin kimin için yazıldığı
- İlaçlar: İlaç tipinde nesneleri tutan bir dizi

İlaçlar ayrı bir sınıfta tutuldu, çünkü her ilacın adı, içeriği, kullanıldığı hastalık gibi kendine ait pek çok bilgisi vardır. **İlaç** sınıfına pratik olarak yalnız bir nitelik yazıldı: ad. Asistan, bu sınıfın gerçekte daha ayrıntılandırılacağını söyledi.

Reçete ile İlaç arasındaki ilişki için asistan Java'da görülen üç ilişki türünü hatırlattı. En temel ilişki association'dır. İlişki biraz daha kuvvetlendiğinde aggregation (bir araya getirilme) kullanılır. Bir reçetede birden çok ilaç vardır ve bu ilaçlar o reçeteye özel olarak bir araya getirilmiştir: Bir reçetede dört ilaç, başka bir reçetede üç ilaç vardır. Bu yüzden aggregation seçildi. Violet'taki ifadeyle "İlaç, Reçete'nin bir aggregate'idir" (is an aggregate of). Aggregation, bütün tarafında içi boş bir elmasla çizilir; elmasın ucu bütündür (Reçete), öbür taraf küçük parçalardır (İlaç).

Asistan, burada düz bir bire çok association kurmanın da yanlış olmadığını, ama reçeteye özel bir gruplama olduğu için aggregation'ın daha doğru bir tasarım olduğunu söyledi.

Sonra iki association daha kuruldu:

- Hasta 1 – * Reçete: Bir hasta adına birden çok reçete düzenlenmiş olabilir.
- Hekim 1 – * Reçete: Bir hekim birden çok reçete düzenlemiş olabilir.

5.6 Hastane, Bölüm ve Çalışan: composition ve aggregation

Şemanın sağ tarafına bir hastane, bölümleri ve bölümlerde çalışanlar eklendi.

- **Hastane:** ad ve kuruluş tarihi.
- **Bölüm:** Hastanenin her branşı (kadın doğum, kulak burun boğaz gibi). Nitelikleri: bölüm adı ve bu bölüme ait "çalışanlar" adlı, Çalışan tipinde nesneleri tutan bir dizi.

Asistan, ilişki türü seçilirken ilişkinin kuvvetine bakıldığını anlattı. Reçete–İlaç ilişkisinde reçete silinse de ilaçlar var olmaya devam eder; ilaç nesneleri tanımlı olarak durur. Hastane–Bölüm ilişkisi ise daha da kuvvetlidir: Hastane olmazsa bölüm diye bir şey de olmaz. Bölüm doğrudan o hastaneye aittir; hastane gidince bölüm de gitmelidir. Böyle bir mantıkta composition kurulur. Composition'da elmasın içi doludur. Bölüm nesneleri vardır, ama tek başlarına var olamazlar; yalnız hastanenin altında, onun bir parçası olarak oluşturulurlar. Bu da bire çok bir ilişkidir: Elmasın ucu 1 tarafında, Hastane'dedir; bir hastanenin birden çok bölümü vardır.

Asistan "gizli bilgi" kuralını burada tekrarladı: Hastane'de aslında bölümler adlı bir dizi tutulacaktır, ama bu nitelik yazılmadı, çünkü composition ilişkisi bunu zaten gösterir.

Öbür tarafta Bölüm'e çalışanlar dizisi yazılmıştı. Bölüm ile Çalışan arasındaki ilişki için aggregation seçildi. Gerekçe: Çalışan başlı başına var olabilen bir nesnedir. Bir bölüm kapatıldığında, örneğin o bölümün hemşiresi başka bir bölüme verilir; çalışana bir şey olmaz. İlişki association'dan biraz daha kuvvetlidir, ama composition'daki gibi tam aitlik ve başka yerde kullanılmama durumu yoktur.

> **Hocanın vurgusu:** Parça, bütün silinse de var olmaya devam ediyorsa aggregation (içi boş elmas) kullanılır. Bütün olmadan parça da olamıyorsa composition (içi dolu elmas) kullanılır.

Asistan son olarak şemada hiç ilişkisi gösterilmemiş, "oksuz" bir sınıf kalmadığını kontrol etti.

5.7 Şemanın metin hâli

```
Erisim
  - ad
  - soyad
  - kimlikNo
  - telefonlar : dizi
  - adres : String

Calisan
  - gorevBaslamaTarihi
  - gorevdenAyrilmaTarihi
  - erisim : Erisim
Calisan 1 ------ 1 Erisim

Hekim, Hemsire, GuvenlikGorevlisi, Hizmetli ----|> Calisan   (kalıtım)
Hekim
  - diplomaNo : int
Bashekim ----|> Hekim                                    (kalıtım)

Hasta 1 ------ 1 Erisim        (erisim niteliği yazılmadı: "gizli bilgi")
Hasta 1 ------ * Randevu

Recete
  - no
```

```
  - duzenleyen : Hekim
  - tarih
  - hasta : Hasta
  - ilaclar : Ilac[]
Hasta 1 ------ * Recete
Hekim 1 ------ * Recete
Recete <>----- * Ilac          (aggregation, içi boş elmas Recete tarafında)

Ilac
  - ad

Hastane
  - ad
  - kurulusTarihi
Hastane <#>---- * Bolum        (composition, içi dolu elmas Hastane tarafında)

Bolum
  - bolumAdi
  - calisanlar : Calisan[]
Bolum <>------- Calisan        (aggregation)
```

Asistan dersi şöyle bağladı: Bugünkü uygulamada kullanım şeması çizildi, senaryo yazıldı ve sınıf şeması oluşturuldu.

6. Ders Sonrası Notlar

Ders sonrası not: Asistan `include` için "bu, bundan önce olmuş olmalıdır" dedi ve iki yerde bu anlamda kullandı. UML'de doğrusu şudur: `include`, temel use case her çalıştığında içerilen use case'in de o akışın bir parçası olarak çalışmasıdır. Ortak bir adımı ayırmak için kullanılır. Zaman sırası ya da ön koşul `include` ile değil, senaryonun ön koşul alanıyla gösterilir.

- Tedavi bilgilerini görüntülemek, tedavi bilgisi girişini çalıştırmaz. "Tedavi bilgisi girilmiş olmalı", Tedavi Bilgileri Görüntüleme senaryosunun ön koşuludur; bu iki use case arasında `include` olmamalıdır.
- Hasta Kaydı ve Ücret Ödeme, iki ayrı aktörün iki ayrı işidir. "Ödeme alınmış olmalı", Hasta Kaydı senaryosunun ön koşulu olarak yazılmalıdır.

Ders sonrası not: Asistan * çokluğu için "bir veya birden çok" dedi. UML'de *, `0..*` demektir: sıfır ya da daha çok. "En az bir" için `1..*` yazılır.

Ders sonrası not: Tedavi Bilgisi Girişi senaryosunda hekim ilaçları **dozlarıyla** giriyor, ama sınıf şemasında dozu tutan bir nitelik yok. Doz bir ilacın değil, o reçetedeki kalemin özelliğidir; aynı ilaç farklı reçetelerde farklı dozla yazılır. Daha doğru bir model, Reçete ile İlaç arasına doz ve kullanım bilgisini tutan bir ara sınıf (ör. ReçeteKalemi) koymaktır.

Ders sonrası not: Diploma numarası `int` olarak tanımlandı. Diploma ve kimlik numaraları hesaplamada kullanılmaz; başında 0 ya da harf bulunabilir. Bu tür numaraları `String` olarak tutmak daha güvenlidir.

Ek bilgi (derste anlatılmadı): Asistanın "gizli bilgi" (hidden information) dediği kavram yerleşik bir UML terimi değildir. UML'de bir ilişkinin ucu, o uçtaki rol adıyla zaten bir niteliği temsil eder. Aynı bilgiyi hem ilişki hem nitelik olarak yazmak gereksiz tekrardır; asistanın önerdiği gibi tek biçimde göstermek yeterlidir. Derste Çalışan'da "erişim" niteliği hem yazıldı hem ilişkiyle gösterildi, Hasta'da ise yalnız ilişkiyle gösterildi; iki gösterim de aynı bilgiyi taşır.

7. Örnek Sorular

Bu derste ekrandan kare yoktur. Aşağıdaki sorular asistanın ders sırasında öğrencilere sözlü olarak sorduğu sorulardır.

Soru 1 (derste sözlü, 00:21:46): Kullanım şemasında hasta ve kayıt görevlisinden sonra sırada en önemli aktör hangisidir?

> **Çözüm (derste verildi):** Hekim (doktor). Hekim; hasta bilgisi görüntüleme, tedavi bilgisi girişi ve randevu görüntüleme işlerini yapar.

Soru 2 (derste sözlü, 00:23:34): Hastanın Tedavi Bilgileri Görüntüleme use case'i ile hekimin Tedavi Bilgisi Girişi use case'i arasındaki ilişki `extend` mi, `include` mu?

> **Çözüm (derste verildi):** Derste cevap `include` olarak verildi: "Tedavi bilgisi görüntüleme, tedavi bilgisi girişini include eder; çünkü hastanın görüntüleyebilmesi için önce bilginin girilmiş olması gerekir."
>
> **Ders sonrası not:** Bu gerekçe bir ön koşul ilişkisidir; UML'de `include` ile gösterilmez (bkz. Bölüm 6). Doğru gösterim, iki use case arasında ilişki çizmeden, "tedavi bilgisi girilmiş olmalı" koşulunu Tedavi Bilgileri Görüntüleme senaryosunun ön koşuluna yazmaktır. Yine de iki seçenek arasında seçim yapılacaksa `extend` uygun değildir, çünkü görüntüleme, girişin alternatif bir akışı değildir.

Soru 3 (derste sözlü, 00:39:37): Ücret Ödeme kullanım senaryosunda birincil aktör kimdir?

> **Çözüm (derste verildi):** Hasta.

Soru 4 (derste sözlü, 00:40:12): Ücret Ödeme senaryosunda hastadan başka hangi aktör ilgilidir ve beklentisi nedir?

> **Çözüm (derste verildi):** Kayıt görevlisi. Hastanın ödemesini alıp sisteme işlemek ister. Hastanın beklentisi ise ödemesini yapıp muayene olmaktır.

Dersin Toparlaması

Bu uygulama dersinde bir polikliniğin hasta ve çalışan yönetimi yazılımı için üç analiz çıktısı hazırlandı. Kullanım şemasında beş aktör (Hasta, Kayıt Görevlisi, Hekim, Başhekim, Güvenlik Görevlisi), bunların etkileşimde olduğu use case'ler, `include` ve `extend` ilişkileri ve Başhekim'in Hekim'den genelleştirilmesi gösterildi. Tedavi Bilgisi Girişi ve Ücret Ödeme için ayrıntılı kullanım senaryoları yazıldı; bir senaryoda birincil aktör, ilgililer ve beklentileri, ön ve son koşullar, ana akış ve 2a, 2b, 4a gibi numaralanan alternatif akışlar bulunması gerektiği vurgulandı. Alternatif akışların kullanım şemasında `extend` olarak da gösterilmesi gerektiği belirtildi.

Sınıf şemasında ortak kişi bilgileri ayrı bir Erişim sınıfına alındı, çalışan türleri Çalışan'dan, Başhekim de Hekim'den kalıtımla türetildi. Reçete–İlaç ve Bölüm–Çalışan için aggregation, Hastane–Bölüm için composition seçildi; ayrım, parçanın bütün olmadan var olup olamayacağına göre yapıldı. Analist bakışıyla metotlar yazılmadı ve ilişkiden anlaşılan bilgiler nitelik olarak tekrar edilmedi. Bu uygulama, dönem boyunca görülen nesneye yönelik analiz ve tasarım şemalarının vizeden önce bir arada tekrarı olarak yapıldı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.